Zero Counts

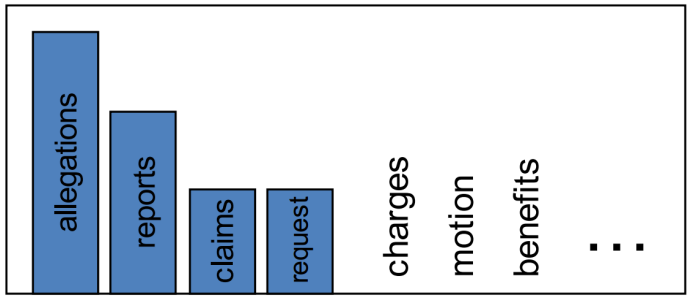
- Training set:
  - … denied the allegations
  - … denied the reports
  - … denied the claims
  - … denied the request
- Test set:
  - … denied the offer
  - … denied the loan

$$q_{ML}(offer|the, denied)$$

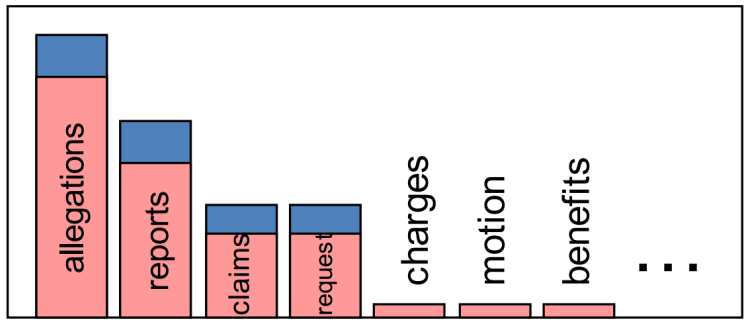
Zero Counts

- But, zero transition probabilities mean that the whole sentence receives a 0 probability!
- Therefore, zero probability estimates are generally avoided
- Methods for assigning probability mass to rare events are often called *smoothing*
  - Some have statistical motivation, others don't

$P(offer|the, denied)$
3 allegations
2 reports
1 claims
1 request
7 total



$P(offer|the, denied)$
2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



Add-δ (Laplace) Smoothing

- Classic solution: add counts
  - If *q(w)* is some distribution over words *w* then the smoothed $q_{add\text{-}\delta}$ *(w)* is

$$q_{add-\delta}(w) = \frac{c(w) + \delta}{\sum_{w'}(c(w') + \delta)} = \frac{c(w) + \delta}{c() + \delta|\mathcal{V}|}$$

- Problem: doesn't work well in practice for LMs

n-gram Smoothing

- The problem is not only zero counts
  - In general, where the event conditioned on has a low probability, the maximum likelihood estimator is unreliable
- Estimating the probability of an n-gram is a classical task in NLP
  - Not only in language models
  - For instance, knowing the common n-grams a word participates in can be telling in terms of its meaning
- The problem becomes worse as n increases
  - The denominator grows exponentially with n

Discounting Methods

- Maximum likelihood estimates tend to be too high for low count items:

x	Count(x)	$q_{\text{ML}}(w_i \mid w_{i-1})$
X	48	
X , dog	15	15/48
X , woman	11	11/48
X , man	10	10/48
X , park	5	5/48
X , job	2	2/48
X , telescope	1	1/48
X , manual	1	1/48
X , afternoon	1	1/48
X , country	1	1/48
X , street	1	1/48

Discounting Methods

- One simple way to handle this is to discount all counts by a constant term: (say, 0.5)

- But what do we do with the missing probability mass?

x	Count(x)	Count$^*(x)$	$\frac{\text{Count}^*(x)}{\text{Count(the)}}$
X	48		
X, dog	15	14.5	14.5/48
X, woman	11	10.5	10.5/48
X, man	10	9.5	9.5/48
X, park	5	4.5	4.5/48
X, job	2	1.5	1.5/48
X, telescope	1	0.5	0.5/48
X, manual	1	0.5	0.5/48
X, afternoon	1	0.5	0.5/48
X, country	1	0.5	0.5/48
X, street	1	0.5	0.5/48

Discounting Methods

- In a bigram model, the missing probability mass for a preceding word w_{i-1} is:

$$\lambda(w_{i-1}) = 1 - \sum_{w} \frac{c^*(w_{i-1}, w)}{c(w_{i-1})}$$

- In our example: $\lambda(w_{i-1}) = 10 \cdot 0.5/48$
- We will distribute *this* mass (also) to bigrams with probability 0
  - In fact, $P(w_i)$ will be non-zero for every word in the training data

- But how?

Discounting + Unigram Interpolation

- First guess: smooth by reducing some fixed quantity from the bi-gram count, and interpolate with the unigram estimate:

discounted bigram Interpolation weight

$$P_{KN}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{CONTINUATION}(w_i)$$

unigram

Kneser-Ney Smoothing

- Better estimate for the probabilities of unigrams:
  - What's the next word: *I can't see without my reading*_____?
  - "Francisco" is more common than "glasses"
  - … but "Francisco" always follows "San"
- So we want to interpolate with a measure of "How likely is *w* to appear as a novel continuation?"
  - For each word, count the number of bigram types it completes
  - Every bigram type was a novel continuation the first time it was seen

$$P_{CONTINUATION}(w) \propto |\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

Kneser-Ney Smoothing

- Properly normalized:

$$P_{CONTINUATION}(w) = \frac{|\{w_{i-1} : c(w_{i-1}, w) > 0\}|}{\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}|}$$

- A frequent word (“Francisco”) occurring in only one context (“San”) will have a low continuation probability

Kneser-Ney Smoothing

$$P_{KN}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{CONTINUATION}(w_i)$$

- λ is a normalizing constant; assume d≤ 1.
- The probability mass we've discounted:

$$\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} \left| \{w : c(w_{i-1}, w) > 0\} \right|$$